

2.2 Typumwandlung

Yusuf hat den Code rechts entworfen. Er führt das Programm aus und erhält die Bildschirmausgaben. Zwei fehlen.

Vorhersage: Trage die fehlenden Ausgaben ein und erkläre in einem Satz, wie sie zustande kommen.

<code>println(3 + 5);</code>	Ausgabe: 8
<code>println("3" + "5");</code>	Ausgabe: _____
<code>println("Test" + 5 + 8);</code>	Ausgabe: _____
<code>println("Test" + (5 + 8));</code>	Ausgabe: Test13

Aufgabe 1 Projekt Intro

a) Ausführen und vergleichen. b) Eigene Kombinationen (TODO 1).

c) `0 + 'A'` (TODO 2): Ausgabe _____ Grund: _____

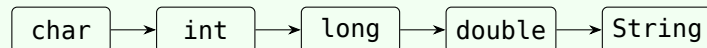
Implizite Typumwandlung

Bei `"Test" + 5` findet eine so genannte implizite _____ statt: Die Zahl 5, üblicherweise vom Datentyp `int`, wird als _____ (Datentyp von Text) interpretiert.

Das Plus-Zeichen ist dann kein Rechenoperator, sondern eine _____ (Aneinanderhängen) der Zeichen.

Die implizite Typumwandlung ist in Java nur möglich, wenn dabei _____

Informationen verloren gehen. Daher gilt in Java folgende Kette:



Jeder Datentyp kann direkt in die weiter rechts stehenden umgewandelt werden.

Prüfe im Projekt: `double d = 5; _____ int i = 5.0; _____`

Anmerkung: Zu String wandelt Java nur beim Verketteten mit `+` um (`" " + 5`). Eine Zuweisung wie

`String s = 5;` meldet der Compiler als Fehler.

Aufgabe 2

Entferne TODO 3: `int x = 3.7;` Fehlermeldung (sinngemäß) und Grund:

Wie könnte man Java sagen „ich nehme den Verlust in Kauf“? Idee:

Explizite Typumwandlung

Der gewünschte Zieldatentyp wird in runden Klammern vor den umzuwandelnden Wert geschrieben (englisch: *cast*):

```
int i = (int) 3.7;    char c = (char) 65;    int k = (int) 4294967298L;
```

Typumwandlung	Startwert	Endwert
double → int	3.0	
double → int	3.7	
double → int	−3.7	
int → char	65	
int → char	66	
char → int	'a'	
long → int	300	
long → int	4294967298	
Eigene Beispiele (vorher Endwert vermuten, dann im Projekt prüfen):		

Achtung: Hierbei kann es zu _____ (nicht alles bleibt erhalten) kommen.

Nachkommastellen werden _____, nicht gerundet.